Grouper: Scheduling Groups for Multi-Tenant Microsecond-Scale Microservices

Koosha Kazemi
Sharif University of Technology
koosha.kazemi24@sharif.edu

Mohammad Siavashi*
KTH Royal Institute of Technology
siavashi@kth.se

Ahmad Siavashi†
Independent Researcher
a.siavashi@outlook.com

Mohammad Izadi
Sharif University of Technology
izadi@sharif.edu

Abstract

Microsecond-scale core allocation makes colocating latency-critical services with batch work worthwhile. A thread that finds no work parks within microseconds and its core goes to a batch task. Putting one back costs ~18 μs, as the allocator must discover that a core is wanted and then take it from the batch task holding it. A monolith pays that tax once per request, a microservice chain pays it at every hop in both directions, and a multi-tenant host multiplies it again, because every tenant's hops queue at the same allocator. On our port of DeathStarBench's `hotelReservation`, going from two tenants to ten takes a hop from 39 to 222 μs and a 10-RPC path's median from 456 to 2,445 μs, a fivefold degradation even though no tenant's own load changed.

We introduce Grouper and the *scheduling group*, a set of isolated runtimes that the allocator treats as one allocation and accounting unit, whose members may hand cores directly to one another. A service sending an RPC donates its core to the peer through an unprivileged kernel fast path, so the core follows the request through the call graph. The allocator retains control through reconciliation, core-addressed revocation and a pooled budget but leaves the critical path; its load falls from $\Theta(R{\cdot}H)$ to $\Theta(R)$ in request rate R and hop count H. Over a grid of two to ten tenants at 1,000–30,000 requests per second each, Grouper outperforms Caladan (the allocator Junction also builds on) and Linux by up to 7.9× and 3.4× at the median and 4.1× and 14.2× at the tail, and leaves batch work more throughput than Caladan at over 70% of load points.

CCS Concepts: • **Software and its engineering** → **Operating systems**; • **Networks** → *Data center networks.*

Keywords: datacenter scheduling, core allocation, microservices, kernel bypass, tail latency, colocation

1 Introduction

Datacenter operators want latency-critical (LC) services that respond in microseconds [5], and enough batch work colocated with them that the machines are not mostly idle [10, 30, 42]. Those goals pull against each other. The line of work that made this practical (IX [6], ZygOS [54], Shinjuku [33], Shenango [50], Caladan [21], Junction [19]) moves core allocation out of the kernel scheduler into a dedicated userspace allocator that reallocates far faster than Linux can, harvesting a core as soon as its owner runs out of work and handing it to a best-effort (BE) task. Every one of them places a core **by observation**. The allocator polls, discovers that some application is congested, and moves a core to it, which is the only option available when the applications are unrelated.

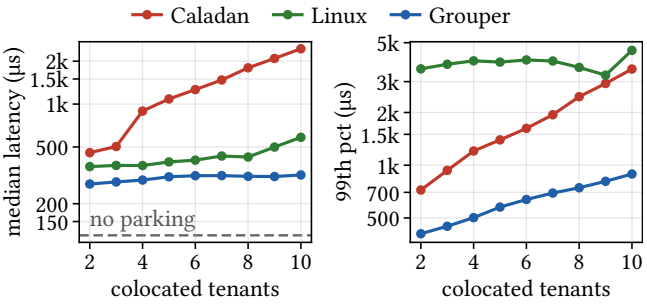


Figure 1. Adding neighbours, not load. Nine-service `hotelReservation` deployments share one 72-core machine with a batch task, each offered a fixed 20,000 requests per second. From left to right, no tenant's call graph, hop count, per-hop compute or request rate changes, only how many other tenants share the machine. Median (left) and 99th percentile (right) of the 10-RPC `search` path. The blue curve in both panels is Grouper. Note the log axes.

Harvested cores pay a "parking tax". Waking a runtime whose threads are all parked is a distributed operation costing ~18 μs when a BE task holds the core and must be made to cede.

A monolithic server pays it once per request. A core is granted on arrival and the request keeps it for its whole lifetime, calling functions sequentially inside one address space. A microservice deployment pays it at each *hop*, and on both legs of each hop. By the time a reply comes back the caller has exhausted its spin window and parked, so the reply must wake it again, while the compute at each stop is often smaller than the cost of arriving there, a sub-microsecond

*Work done while at Iran University of Science and Technology.
†Work done while at Amirkabir University of Technology.

hash lookup or a field rewrite. The regime in which this hurts is the one production runs in. Microservices are provisioned with headroom and run at low to moderate utilization, where inter-arrival spacing exceeds the spin window.

The penalty extends to processor cache locality. Allocator-mediated schedulers blindly harvest cores mid-request when a thread yields: an intervening batch task evicts cache warmth, while later wakes frequently land on another core, incurring cold misses and cross-core coherence traffic on the request's critical path.

Multi-tenancy is what turns the tax into a collapse. Figure 1 shows it, with the same per-tenant load throughout and only the number of deployments sharing the machine increasing. Over that sweep the median of a 10-RPC path grows 5.4× and its 99th percentile 4.9×. The dashed line is what that same path costs when kthreads never park; the difference is the machine failing to put a core where the request already is. Observation is centralized. One allocator must rank every runtime to decide which deserves a core, so its reaction time is set by how many runtimes there are, not by how much work they have. As services multiply, the scan that produces the ranking stops fitting in its own period and runs on every iteration, stretching the loop every grant waits on.

Most of that discovery is unnecessary. The information the allocator polls for is already in the application's dataflow, and only the application has it. A service that has finished its part of a request and is sending an RPC knows which peer needs a core next, and that it is itself about to stop needing the one it is standing on. If the two processes are mutually trusted, the sender can hand its core over, and there is nothing left to discover.

Grouper builds on that. A **scheduling group** is a set of isolated runtimes, with separate address spaces, failure domains and binaries, that the allocator treats as one allocation and accounting unit, and whose members are mutually authorized to hand each other cores through an unprivileged kernel fast path in about 4 μs, with no allocator round trip. The allocator is involved once per request, at the edge where the request enters the group; every interior hop is a transfer it never sees.

Making that safe is most of the work. Two things break as cores move without the allocator's knowledge: accounting drifts, and thread-targeted revocation preempts the wrong occupant. Grouper responds with per-iteration reconciliation and core-addressed revocation. Direct handoff also avoids NIC round trips via a shared-memory datapath delivered synchronously with the donation, leaving an interior hop free of receive queues, hashes or steering decisions. Because execution stays on the same physical core, the handoff acts as a cache-preserving execution migration, retaining processor cache and translation locality across process boundaries.

A group that passes one core along a call chain makes the *request*, not the task, the unit of host scheduling, and we state that as an invariant. **At every instant, a request in flight inside a scheduling group occupies exactly one core, and no other request occupies it.** Under it kthreads stop being interchangeable workers that steal from one another's runqueues and become deterministic landing *pads* for specific request *lanes*, which gives the chain the locality of a single-threaded monolith, gives the group an explicit concurrency bound, and yields edge admission control.

We contribute the scheduling group abstraction and its trust model (§3.1); the one-core-per-request invariant and the group-wide lane naming that makes it enforceable across address spaces and yields edge admission control (§3.4); unprivileged directed handoff together with the reconciliation that keeps a central allocator's accounting exact (§3.2–§3.3); and an evaluation of Grouper versus Caladan and Linux.

2 Background and Motivation

2.1 Core harvesting and the parking tax

A core-allocating host scheduler divides a machine between a privileged control plane and unprivileged runtimes. The **allocator** owns the cores. It interleaves a sub-microsecond fast pass with a periodic **slow pass** that scans every runtime's queues to estimate congestion and decide allocations, and grants or revokes a core through a kernel module that wakes a specific thread on a specific core. Each runtime is a userspace scheduler running user-level threads (*uthreads*) on kernel threads (*kthreads*), one per granted core [2, 56]. The structure is common to this class; we measure Caladan [21], whose slow pass is scheduled every 10 μs.

The harvesting rule is aggressive. A kthread that finds its runqueue, ingress queue and timer wheel empty spins for 2 μs and then parks, which returns a core to batch work almost as soon as it is genuinely idle, and means a service idle for even a few tens of microseconds between requests is reliably found with all of its kthreads asleep. Waking one back up is a distributed operation. The allocator must *notice* the work; rank congested runtimes and choose a core; if a batch task occupies that core, interrupt it and wait for it to cede; and only then wake the target kthread there. Table 1 breaks the parking tax down.

2.2 The tax is per hop, and its price is set by the neighbours

A monolithic server amortizes one wake over an entire request. A microservice chain does not. Each RPC crosses a process boundary, and each crossing finds a service whose kthreads have parked, on the way down because the callee has been idle since its last request, and on the way back because the caller parked while blocked on the reply. A chain that makes H RPCs therefore incurs roughly $2H$ wakes.

We ported DeathStarBench's `hotelReservation` [23] onto such a runtime to measure this on an actual call graph. Its `search` path is four levels deep (client → frontend →

Table 1. Where a wake goes (μs, p50). One hop, 70 schedulable cores, vfio directpath, x264 on every core the service does not hold.

stage	idle core	BE on the core
detect arrival	0.1	0.2
choose core	0.4	0.5
IPI + signal	0.0	5.7
cede	0.0	1.3
context switch	5.6	9.9
total	**6.1**	**17.6**

search → rate → kv) for a total of **10 RPCs**, and non-empty searches are ~31% of the standard mix. Weighting the whole mix gives ≈**5.6 RPCs per request**, and the compute at each stop is often a single hash lookup and a field copy, well under a microsecond. This is a workload where the scheduler, not the application, sets the latency [59].

That shape is representative. Production traces from Alibaba [45] find an average call-graph depth of 4.27, with many deep graphs reducing to a single long chain; Google's Online Boutique, which §5.6 measures, is a shipped instance of that tail, 11.8 RPCs from a graph three levels deep. Every level is a boundary crossing that finds a parked service, which is why even modest depth pays a large tax [14, 68].

Scheduling is most of this latency, and what raises it is the neighbours rather than the load (Figure 1). Six times the offered load per tenant costs the allocator 1.7×; holding each tenant's rate fixed and adding five times as many tenants costs it 5.4×. Nothing about any individual request has changed. What changes is the allocator's reaction time. With dozens of colocated services, the allocator's slow pass must inspect hundreds of kthread queues; when this scan exceeds 10 μs, it stops being amortized and runs on *every* loop iteration. The dataplane loop period blows up from sub-microsecond times to tens of microseconds, delaying every core grant and making per-hop delivery surge from 38.6 μs at two tenants to 221.6 μs at ten (§5.5). Making the polling cheaper does not fix this (§2.3). **The bottleneck is the per-hop acquisition itself**, whose decision loop stretches with every tenant on the machine.

2.3 Why the obvious remedies fail

Spin longer. A runtime can spin longer before parking [36], or pin a kthread that never parks. Both shrink that tax, and pinning removes it outright, but both charge it to the same account. A spinning core is held whether or not a request arrives, and every one of them is a core the batch task cannot have. The reservation is per service, so even a small microservice deployment fits few tenants on a machine and leaves the batch task very little. §5.7 prices the whole sweep.

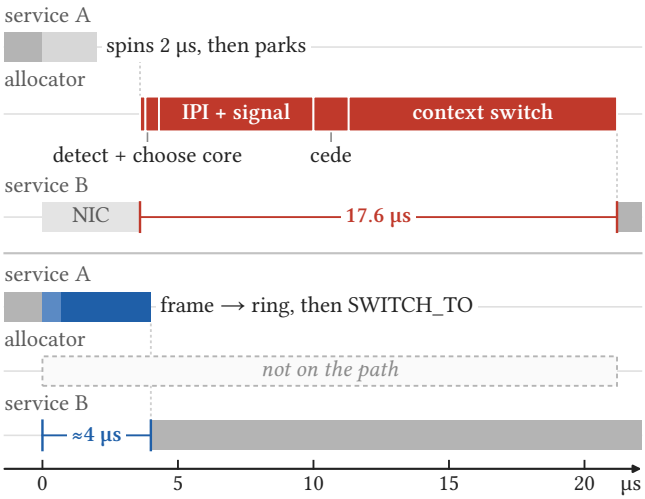


Figure 2. The cost of one hop, with and without the allocator. Park-and-wake on top, with Table 1's stages in proportion. Directed handoff into the receiver's ring on the bottom.

Make polling cheaper. Junction [19] scales the allocator to thousands of instances with a NIC event queue that arms idle receive queues instead of polling them and a 16 μs hierarchical timer wheel that skips idle runtimes. This helps at lower load but converges on the default as the machine fills. Scanning less often would help too, but the slow pass supplies the signal that ranks runtimes for a core, the estimates that decide who may share one and the queueing delays the policy compares, so widening its interval coarsens all of them at once, precisely when tenants are most numerous and interference mitigation matters most. The hops have to stop arriving at that scan.

Table 2 is where this leaves the three systems we measure, and the two baselines fail in opposite ways. Linux has no central allocator, so an interior hop costs a tenant the same whatever its neighbours do, but it cannot take a core back from a batch task in microseconds and pays for that in the tail. Caladan (and Junction, which vendors the same allocator) can, but every hop is then a request to one machine-wide decision maker whose reaction time is set by how many runtimes are asking. Only Grouper provides both.

3 Design

A scheduling group is a set of N runtimes that declare, in their configuration files, that they belong to the same named group. The allocator treats the group as one allocation and accounting unit, with one pooled core budget and one shared region of state, and authorizes every member's kernel threads to hand a core directly to any other member's. Members remain separate processes with separate address spaces, failure domains and binaries, and nothing about the application programming model changes. The design must do four things

Table 2. What each system offers a microservice chain.

System	Reclaims a core from batch in μs	Interference control	Interior hop off the NIC	Hop cost flat in tenants	Core follows the request	Admission control	search p50 (μs)	search p99 (ms)
Linux	✗	✗	✓	✓	✗	✗	363–1,057	3.3–11.0
Caladan / Junction	✓	✓	✗	✗	✗	✗	447–2,565	0.58–3.86
Grouper	✓	✓	✓	✓	✓	✓	271–371	0.37–1.02

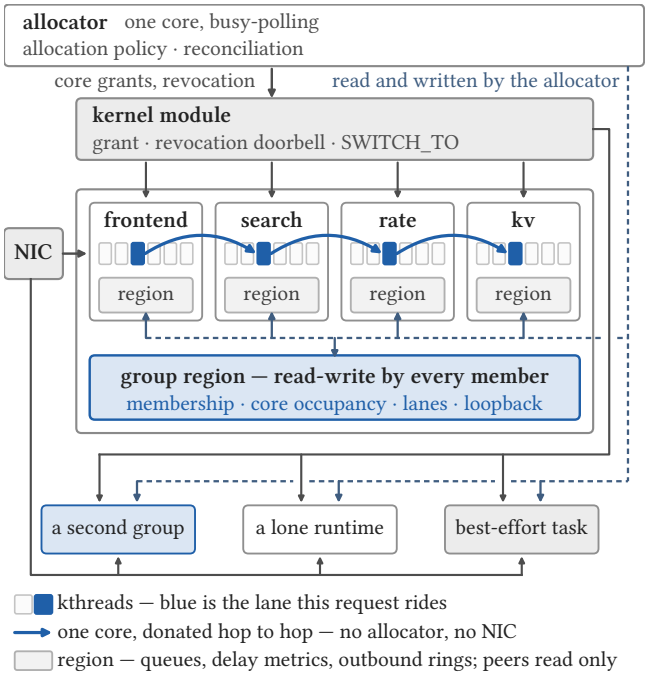


Figure 3. Grouper's system architecture. Isolated members share a pooled core budget and one group region. An interior hop leaves the frame in the ring its peer drains and donates the core, so neither the allocator nor the NIC is on its path. The allocator is the fallback for any hop that is not donated. The same allocator and the same NIC serve the rest of the machine (another group, an ungrouped runtime, a best-effort task).

at once. It must move a core between address spaces in microseconds with no allocator round trip (§3.2); keep the allocator's accounting exact and its preemption working when it no longer knows who occupies a core (§3.3, §3.6); keep process isolation, so that no member holds a writable mapping of another's memory; and fall back to park-and-wake whenever group state is absent or inconsistent (§3.1). Figure 2 contrasts a hop with and without the allocator on its path; Figure 3 shows the resulting architecture.

3.1 Trust, lifecycle and the fallback rule

Trust model. A group encapsulates *one tenant's application*, the same team's services, deployed together, already able to invoke each other. It never spans tenants and is not a security boundary between mutually distrusting parties, which is what makes mutual donation sound. Isolation is preserved across process boundaries. Loopback rings are mapped asymmetrically and read-only, and handoff authority is granted by the privileged allocator rather than claimed by members. A member's authority also stops at the group. A donation moves a core the group already holds, the pooled budget is enforced by the allocator when it grants (§3.7), and a member that will not yield is preempted by the same core-addressed revocation as any other occupant (§3.6), so nothing a member does reaches another tenant. A group is host-local as well. A deployment spanning machines runs one group per host, with ordinary networking between them.

Lifecycle. A group is formed, made ready, and dissolved by the allocator, and outside that window its members are ordinary runtimes. Two configuration lines make a runtime a member, a group name and an expected member count N. The allocator allocates a shared region on first registration and passes it to each member as it attaches; until all N have arrived the group is not `ready`. When the last registers, the allocator authorizes every member thread in the kernel module, publishes the lane count (§3.4) and sets `ready`. A member that exits or crashes dissolves the group rather than repairing it, and the survivors fall back to park-and-wake, with no error path.

The fallback rule. Nothing in the shared region is needed for correctness. Every path that consults it (donation, lane acquisition, loopback transmission, reconciliation) checks readiness first and takes the park-and-wake path if anything is missing or inconsistent. That is what makes group state a *hint layer* over an unmodified allocator, and why a misbehaving or crashed member costs performance rather than correctness.

3.2 Directed handoff

The fast path is the unprivileged `SWITCH_TO` ioctl. A kthread owning a core calls it with a target thread ID, and the kernel module hands the core over. After confirming caller occupancy and that no allocator revocation is pending, it wakes the target, transfers the occupancy record, and yields. Linux then switches directly to the successor on the same physical core (about 4 μs), keeping the core's busy flag set throughout so it never appears idle. Because execution stays on the same core, this migration preserves warm L1/L2 caches, while

Linux's PCID support retains TLB translations across the address-space switch without a full flush. A refused donation is not an error; the caller falls back to ordinary park-and-wake.

3.3 Reconciliation

Handoffs happen inside groups and the allocator never sees them, so its record of who sits on which core goes stale in both directions at once. It still counts the donor as holding the core it gave away, and does not count the recipient as holding anything. Every allocation decision is computed from those counts, so they have to be accurate. Each core's record in the shared region carries a handoff sequence number and the allocator caches a pointer to it, so discovery costs one load per core and is almost always an equality; when it differs, the allocator moves the active-thread counts across. It is discovery and not enforcement. What it cannot resolve, a thread already active elsewhere, because handoffs chain, is left a pass, and a periodic repair concedes any lasting disagreement back to the runtime, which is the authority on whether its own thread is running.

3.4 One core per request

> **At every instant, a request in flight inside a scheduling group occupies exactly one core, and no other request occupies it.**

The invariant restores, to a distributed call chain, the property a monolith gets for free. A request arrives on a core and keeps it until it leaves, because the calls it makes are function calls. Grouper makes the same true across process boundaries, so kthreads cease being interchangeable workers and become deterministic landing pads for particular requests.

Lanes. A *lane* is the group-wide name for the core a request is riding. Lane l means kthread l at every member. A request holding it runs on kthread l wherever it is, sends into ring l of whatever peer it calls, and donates to kthread l (Figure 4). Because the name must mean the same thing everywhere, every member's kthread count is the same.

Changing lanes is not allowed, so **lanes cannot be shared**. Two requests riding lane l would be on kthread l of every service they both reached, which is the double occupancy the invariant forbids. A lane is therefore *booked* on entry and cleared on exit. Two consequences carry the rest of the design. The lane count is a hard concurrency bound, which §3.4.1 turns into admission control; and because only the holder of lane l can send along it, a kthread's supplier is unambiguous with no arbitration. Booking occurs only at the edge. Inside the group a frame arrives through the ring that already names its lane.

Fixed receive queues. A general dataplane must re-steer traffic whenever kthreads park or wake, forcing awake threads to adopt parked queues and churning flow assignments or NIC RSS tables [4, 37, 52]. This adds synchronization overhead on core transitions and breaks request locality. Grouper eliminates steering altogether by permanently binding receive queue i to kthread i. Because the core travels along the request's lane, the core moves to the work rather than steering work onto an awake core (§3.5).

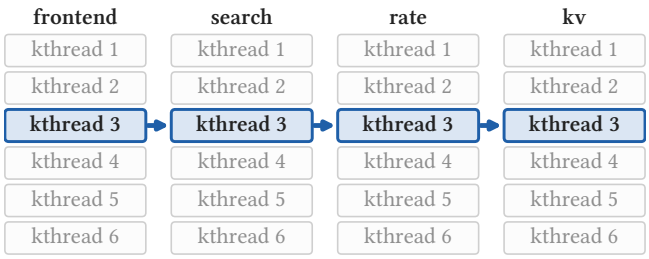


Figure 4. A request's lane along the call graph. Lane l is kthread l at every member; the core travels with the request. The request is shown going through lane 3; a mis-steered packet claims a free lane at the first hop.

First-hop placement. Inbound network packets are steered by NIC RSS, which is unaware of the group's lane bookings. If a packet lands on a kthread whose lane is already booked, the runtime claims an available parked lane, places the request onto that lane's kthread, and donates the arrival core directly to it.

Transparent integration. Applications already choose per-core resources (connection pools, client handles, sharded buffers) through an affinity call returning the current kthread index. Inside a group that call returns the current *lane* instead, so connection l is chosen, its frames go to ring l of the peer, and that ring is drained by the peer's kthread l, the kthread the core was donated to. Unmodified applications keep flow affinity without a line of change.

3.4.1 Admission control at the edge.

When every lane is held, booking fails, and the runtime raises a flag that the application reads once and turns into a protocol-level refusal (an HTTP 503, say). One hook in the edge gateway is the whole application-side cost; the runtime cannot render the refusal itself, because its only means of refusing is to abort the connection, which on a pooled connection destroys every other request sharing it just to turn away one.

A failed booking means the group already has as many requests in flight as it has lanes, and the honest answer is to refuse that one rather than run it on a core another request is riding. A request admitted into an oversubscribed group accumulates latency at every hop and very likely misses its tail SLO *after* burning several hops of work [11, 64], while the cores it would have consumed keep running batch work if it is turned away. It can also make a laneless request *wait* instead; this is off by default.

3.5 Group loopback

Microservices that talk over the NIC pay double PCIe traversals, DMA and packetization [35, 39], and compete with external traffic for NIC resources. Grouper replaces intra-group networking with a shared-memory loopback transport that delivers messages through cache and memory synchronously with the core handoff.

Each ordered pair of members gets one lockless channel per receiving kthread. The descriptor ring lives in the *sender's* region and the consumer writeback in the *receiver's*, so each side writes only its own memory and reads the other's read-only. No member holds a writable mapping of another's address space, as a property of the layout rather than an enforced rule. Because a ring names a (member, kthread) pair, the sender chooses which of the receiver's kthreads picks up the frame with no RSS hash or flow table. That is sound only because the sender then donates its core to that kthread. The send writes the descriptor and pending hint *before* donating, so the frame is in the ring when the receiver resumes. Because handoff runs the receiver on the same physical core, this "hot handoff" serves descriptors and payload directly from local L1/L2 cache. By contrast, allocator-mediated wakes often schedule the receiver on another core or after an intervening batch task, incurring cross-core coherence traffic and cold cache misses. The receiver copies the frame into its address space; the layout also admits zero-copy for larger transfers.

That hint is the recovery path as well. Loopback bypasses hardware NIC queues, so a frame could sit unnoticed if a donation is refused or preempted. The allocator sweeps the hints on its slow pass (§3.8) and wakes any recipient still parked that has frames pending.

3.6 Core-addressed revocation

An allocator revokes a core by writing a sequence number into the queue pointers of the kthread it believes is there, which it can do, because it put them there. With groups, it cannot. Cores change hands without its involvement, so a revocation addressed to a thread reaches the wrong one. Grouper addresses revocation to the **physical core** instead. The allocator increments a cede sequence in the core's record; whoever is executing there notices the mismatch at its next preemption check, yields, and acknowledges by matching the sequence. An arriving kthread also retires any outstanding cede on the core it *lands* on, so a stale revocation cannot immediately re-park whoever just arrived. Because the revocation is durable state rather than a message, the interrupt that makes a compute-bound occupant look at it is only a doorbell. A lost doorbell costs latency, never correctness, and can be rung again without rewriting anything that names a kthread.

Table 3. Allocator dataplane cost. C managed cores, A runtimes on the poll list, K kthreads per runtime, G groups, H RPCs per request, R request rate.

	Allocator-mediated	Grouper
polling (fast pass, per iteration)	$\Theta(C)$	$\Theta(C)$
polling (slow pass, per 10 μs)	$\Theta(A{\cdot}K)$	$\Theta(A{\cdot}K + G)$
work (grants one request causes)	$\Theta(H)$	$\Theta(1)$
work (grants machine-wide)	$\Theta(R{\cdot}H)$	$\Theta(R)$

3.7 Allocation policy

Grouper's allocation policy ranks runtimes by how badly each needs a core and enforces a configured core limit and guarantee for each.

Pooled budget. A group is one entity for provisioning with its own configured limits and guarantees. Its lane count is its pooled budget and the cores held across all of its members can never exceed that budget. A group cannot monopolize a machine by being split into more services, and an operator sizes a tenant rather than each of its services separately.

Per-member ranking. Members are ranked for *urgency* nonetheless by their own active-thread count, not the group's. A member with no cores and a packet waiting is considered congested and promptly served a core by the allocator. Refused donations and fan-out fall back to ordinary park-and-wake, which by definition cannot be donated a core.

3.8 Allocator load

Table 3 separates two things that are easy to mix up. One is what the allocator spends *looking* for work. The other is what a request actually makes it *do*.

The two polling rows are almost unchanged. Reconciliation adds a constant per core rather than a walk (§3.3), and the slow pass gains only $\Theta(G)$ for the loopback sweep, dominated by the pre-existing $\Theta(A{\cdot}K)$.

The two work rows are where the difference lives, and they are not measured by the passes. They are what the passes *initiate*. Placement by observation parks the caller on both legs of every hop and needs an allocation for each, $\Theta(H)$ per request, $\Theta(R{\cdot}H)$ machine-wide. Grouper's members hand the core over instead. A request over `hotelReservation`'s standard mix makes ≈5.6 RPCs and every leg of each is a core transition, ≈9 of them between members which the group takes care of. What remains for the allocator is the edge, and nothing that grows with the depth of the call graph. **Grouper's advantage is that the allocator's load**

stops multiplying by hop count, which is why the group's latency is flat in call-graph depth and in tenant count where an allocator-mediated one is not.

4 Implementation

Grouper reuses the runtime and allocator of Caladan [21], branched from its then-latest commit (`bdb4dde`). Caladan is regularly maintained and is the scheduler of the newer Junction [19]; its allocator is the strongest published instance of the placement-by-observation design of §2.1, and its runtime already provides uthreads, kernel-bypass TCP/IP and the clean preemption of a core that §3.6 rebuilds. Our changes span the kernel module, allocator (the IOKernel) and runtime, adding about 10,500 lines and removing about 200; Table 5 in Appendix A breaks that down. Two ioctls are the whole of the kernel-module change. One is privileged, registering a group's threads, and the other is the unprivileged `SWITCH_TO` of §3.2. The IOKernel gains a registration and region-allocation protocol on its control socket, a reconciliation step in the per-core walk of every dataplane iteration, and §3.7's policy, which budgets a group as one entity while ranking its members individually for urgency. In the runtime the changes cluster at three points. The park path checks for a pending donation before it parks, the send path decides whether a peer is reachable by loopback and arms a donation, and `schedule()` handles first-hop placement and returns lanes. Every path takes the fallback by ordinary control flow rather than via an error handler. We apply a small set of fixes to stock Caladan before measuring (Appendix B); every Caladan number here includes them.

5 Evaluation

5.1 Methodology

Testbed. One two-socket Intel Xeon Platinum 8360Y (Ice Lake, 36 cores per socket × 2 HT, 2.4 GHz), 256 GiB DRAM, Ubuntu 24.04 on Linux 6.8, Mellanox ConnectX-6 Dx. Runtimes, hugepages and the NIC all sit on NUMA node 1. The IOKernel occupies that node's 72 hyperthreads, keeps one physical core, and leaves 70 to allocate. Caladan runs in external directpath with VFIO and EQ arming (MTU 9,000) using the default 25 Gbps limit for its bandwidth subcontroller; we disable the hyperthreading subcontroller, which requires guaranteed physical cores and so admits only 35 services on our 35 cores, and whose symmetric guarantees cancel identically in IAS allocation math anyway. We tune for low latency as recommended, disabling TurboBoost, CPU idle states, frequency scaling and transparent hugepages [38]. Ice Lake has no user interrupts, so Caladan's UINTR path is off; it is Table 1's IPI+signal row, a third of the tax, and does not touch the observation cost that grows with tenant count (§5.5).

Applications. We port DeathStarBench's `hotelReservation` [23] and Google's Online Boutique [25], keeping each service graph, mix, paths, and static data, and replacing gRPC/protobuf [26] with fixed-layout messages. `hotelReservation`'s memcached and MongoDB become an in-memory KV; Boutique has no datastore. `hotelReservation` is nine services and four levels deep, Boutique ten and three. wrk2 [61] offers requests; locust [43] offers *tasks*, two of which issue more than one HTTP request, so a load point of 30,000 is 30,000 RPS per tenant on `hotelReservation` and 36,300 on Boutique. Rates we report are in requests.

Best-effort antagonist. `x264_be` runs 72 independent single-stream x264 encoders [63], one per core, at BE priority. Independence makes every reclaim a genuine preemption (a frame-threaded encoder would yield at its sync points and collapse the tax into Table 1's idle-core wake) and is the usual batch-transcode configuration.

Deployment. A *tenant* is one complete deployment, with its own processes, address range and scheduling group; tenant count ranges over 2–10 (18–90 processes). Each tenant gets six lanes (§3.4). One load generator holds eight cores and round-robins every tenant's frontend.

Arms and runs. Four arms. `Caladan` (with the Appendix B baseline fixes), `Caladan+tw` (idle runtimes move from the poll list onto a timer wheel, the Junction variant), `Grouper`, and `Linux` (batch task at `SCHED_IDLE`, interior traffic over kernel loopback). In three cells (nine tenants at 30,000 RPS and ten at 25,000 and 30,000) the Linux generator can no longer hold the arrival schedule and offers only 72–83% of nominal; those three are excluded from every range we quote for it. Every cell is three interleaved repetitions of a 20 s measurement and we plot the median of the three. Grouper's spread across repetitions is under 2% of its median everywhere in the grid; Caladan's is under 2% from six tenants up and wider below five, where its curve is steep enough that a run lands high or low on a slope rather than on a level. We report `hotelReservation`'s 10-RPC `search` path (31% of the mix) and Boutique's `browseProduct` (57%, 12 RPCs), taking either per deployment and aggregating via a median.

Baseline provenance. The Caladan arms are not the 2020 artifact, but the maintained upstream tree at commit `bdb4dde` (May 2026) [22], which is the tree Junction [19] vendors as its scheduler and runtime [20]. Junction makes kernel bypass practical, with unmodified Linux binaries, a small host attack surface, and thousands of instances per machine, but the allocator it makes practical is the one measured here, and the two mechanisms it scales that allocator with are in this build and in these arms (§2.3). We do not run the services as uProcs in a single instance. That is a different isolation boundary, and a different design point, discussed in §7.

5.2 Latency

The grid crosses tenant count 2…10 with 1,000…30,000 requests per second per tenant, up to 300,000 in total across 90 services on 72 cores against 72 encoders (Figure 5); Figure 1 took a cross-section at 20,000 RPS per tenant.

Caladan's search median rises from 456 μs to 2,445 μs over that line and its 99th percentile from 722 μs to 3,539 μs (5.4× and 4.9×), on a machine whose per-tenant load never varies. Grouper goes from 275 to 318 μs and from 407 to 891 μs (1.2× and 2.2×). The gap widens monotonically in tenant count rather than remaining a constant factor, from 1.7× to 7.7× at the median and 1.8× to 4.0× at the tail, which is the shape §3.8 predicts. What a group removes is $\Theta(R{\cdot}H)$ work at the allocator, and how much there is to remove grows with the number of tenants asking.

Grouper refuses 3.3–5.7% of requests along this line and Caladan none; every latency here is of a served request. Refusal is not what buys the gap. At ten tenants no offered load in the grid puts Caladan where Grouper is. Its median is 2,445 μs at 20,000 RPS per tenant, 1,523 μs at a quarter of that rate and 1,013 μs at a twentieth, against Grouper's 318 μs at the full rate. Appendix C gives Caladan the same bound directly.

Linux is flat where Caladan is not, and its tail is why that is not enough. With no central allocator to queue behind, its median rises only 364 to 585 μs along the same line, beating Caladan at every tenant count, and holds 363–1,057 μs over the rest of the grid. Its 99th percentile does not follow. It runs from 3,547 to 4,524 μs, 3.3–11.0 ms over the grid and 4.0–8.7× Grouper's along this line, because a latency-critical thread that wakes on a machine whose every core is running the batch task waits out a scheduling slice it cannot preempt [44]. The slice is tunable and tuning it does not close the gap. `base_slice_ns` defaults to 2.8 ms here, a figure the kernel scales with core count; sweeping it at two tenants takes the tail to a floor of 1,958 μs at 700 μs and back up to 2,203 at 350, against Grouper's 407 μs in the same cell. Its tail is flat only because it is already high; Grouper is the only arm that is low in both.

Caladan+tw separates polling cost from allocation cost and confirms that the latter is what matters. Unpolling idle runtimes is worth up to 43% at the lightest load in the grid, and more the more tenants there are to unpoll, but it converges on stock as the machine fills. At ten tenants it is still within 13% of stock at the tail and 3.4× Grouper. Making the scan cheaper delays the cost by about an order of magnitude in offered rate; it does not remove it.

5.3 Best-effort throughput

Colocation is the reason these runtimes harvest cores at all [30, 42], so latency bought by starving the batch task is not bought at all. Figure 6 (left) takes the 20,000 RPS cross-section and Figure 7 answers the question over the whole grid. Grouper leaves the encoders more throughput than Caladan in 45 of 63 load points, by up to 37%, within a tenth of it in 51, and less by at most 40% in the rest. Linux tracks Caladan along this line to seven tenants and then falls away, leaving the encoders 123 frames/s at ten against Caladan's 221 and Grouper's 159.

The mechanism is how often the batch task is disturbed. At ten tenants offering 20,000 RPS each, Caladan preempts the encoders 326,000 times a second and Grouper 29,400 (an eleventh as often). Cores are already busy, so the tax is weakest here. At lower RPS more hops park and Caladan preempts considerably more per request; Grouper takes one core at the edge and carries it along the call graph.

5.4 Goodput

Figure 6 (right) shows Grouper's achieved goodput at the 20,000 RPS slice. At 1,000 requests per second per tenant Grouper refuses 0.01% and its tail is already 1.5–2.3× better than Caladan's; at 5,000 it refuses at most 0.09% and is 1.6–3.0× better; at 10,000, under 1%, it is 1.7–4.0× better. The advantage is there at essentially zero refusal, and it grows with how much allocator work there is to remove, not with how much traffic is turned away.

Linux refuses nothing and completes every request at all 63 load points, so its line sits on the offered rate throughout, with Caladan's and Caladan+tw's underneath it below four tenants. What that costs is the tail. Grouper's bound is on concurrency, not on rate. Goodput times mean residency is what a group has in flight. At ten tenants (sixty lanes), `hotelReservation` runs 2.2 and 20.7 requests in flight as the offered rate goes 1,000 and 10,000 per tenant, refusing 0.0% and 0.6%; Online Boutique, whose requests are 2.12× as deep, runs 4.1 and 40.3 and refuses 0.0% and 4.3%. A deeper request has a longer residency (324–400 μs against 207–227), so Boutique approaches the bound at a lower offered rate. Carrying one core along a path bounds how many *requests* a group runs at once, never how many hops each makes (Appendix C).

5.5 The cost of a hop

Figure 8 takes the same 20,000 RPS cross-section. Caladan's interior hop costs 38.6 μs at two tenants and 221.6 at ten (5.7×), over a sweep in which no tenant's rate, call graph, or per-hop work changes; Grouper's costs 20.6 and 19.2 μs, and stays between 19.0 and 26.1 over the full grid. Linux's hop is flat too, 28.4 μs at two tenants and 33.6 at ten (27.8–40.4 over the grid), which is the control that names the mechanism. What stretches under Caladan is one allocator's reaction time, and a system without one does not stretch.

What grows is the acquisition. Table 4 holds total offered load fixed and splits it across more tenants. Under Caladan a hop's delivery cost is not a property of the tenant paying it but of how many independent runtimes are asking at once, and a tenant that has changed nothing watches its own

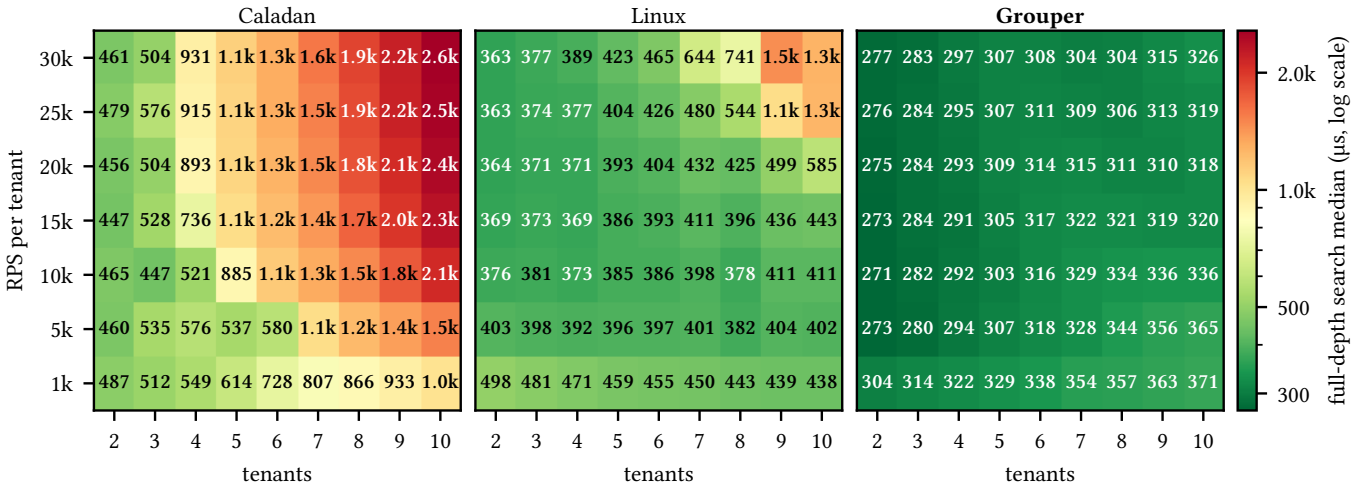


Figure 5. Full-depth search median across the whole grid. One panel per system; rows are load, columns are tenant count.

Table 4. Delivery time of one interior hop (μs) at equal total offered load, split across different numbers of tenants. Each row is one machine load; each half of it is the same requests per second arriving from more independent runtimes.

total offered	split	Caladan	Linux	Grouper	split	Caladan	Linux	Grouper
50,000	2×25,000	38.5	28.1	20.6	10×5,000	163.8	30.8	22.2
60,000	2×30,000	38.2	27.9	20.6	6×10,000	112.3	29.3	21.1
90,000	3×30,000	46.5	28.2	20.6	9×10,000	176.0	30.3	20.9
100,000	4×25,000	74.2	27.8	20.6	10×10,000	211.9	29.9	20.7
150,000	5×30,000	101.2	29.2	20.5	10×15,000	223.5	30.2	19.7

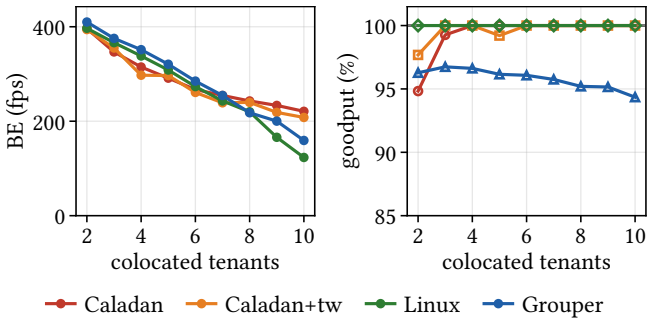


Figure 6. What colocation costs, vs. tenant count at 20,000 RPS per tenant. Best-effort throughput in frames/s on the left (idle baseline 523 frames/s). Goodput as a percentage of offered load on the right, against the dashed offered line; the axis starts at 85%.

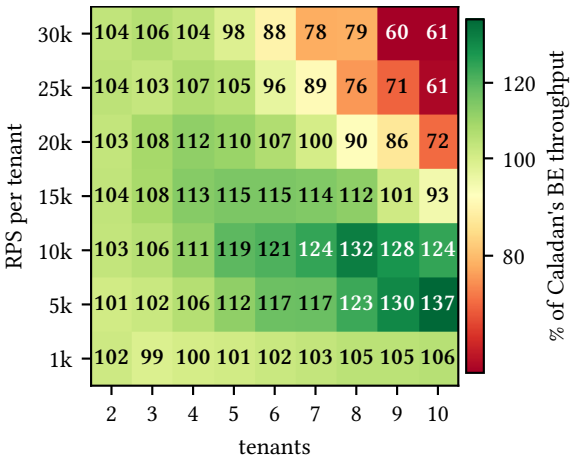


Figure 7. Best-effort throughput across the whole grid. Grouper as a percentage of Caladan's, one cell per tenant count and load point.

per-hop cost quintuple because its neighbours exist. That coupling is what Grouper removes rather than mitigates.

5.6 Doubling the hop count

We ran the grid again against Online Boutique, at 2.12× the RPCs per request, holding every other variable constant (Figure 9). At ten tenants an interior hop costs Caladan 163.8 μs on `hotelReservation` and 158.7 on Online Boutique (the same price, set by how many runtimes are asking rather than by which application asks), and a search request pays it nine times where a browse request pays it eleven. The medians follow, 1,523 and 1,956 μs. Grouper's hop stays roughly

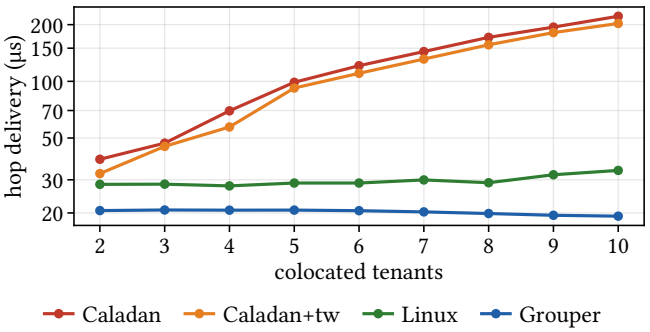


Figure 8. Delivery time of one hop vs. tenant count at 20,000 RPS per tenant. Median interior-hop latency, both legs, visit-weighted.

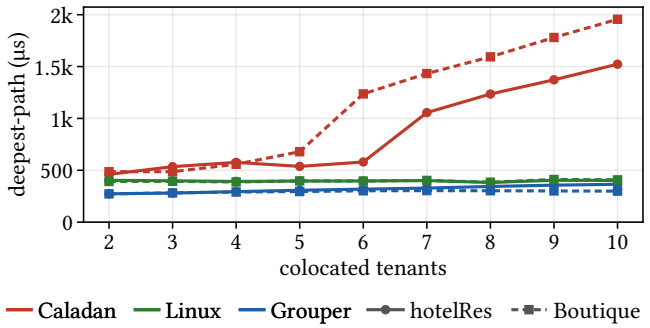


Figure 9. Two applications, three systems at 5,000 RPS per tenant. Deepest-path median for `hotelReservation` (10 RPCs, solid) and Online Boutique (12, dashed). Load chosen so all arms complete ≥99.7%.

constant. Over the whole grid its median holds 271–371 μs on the 10-RPC path and 273–337 on the 12-RPC one, with tails of 370–1,019 and 375–1,318 μs. Linux is flat too, its hop differing by ~4 μs between the two graphs and its median by none, 403 to 402 μs and 392 to 408. What separates those two flat systems is the tail. Linux sits at 3.5–4.0 ms for this cross-section against Grouper's 375–1,318 μs.

5.7 The price of a reserved core

§2.3's remedies both reserve a core across the gap where the request will need it again, charged to the batch task, by pinning a kthread per service or widening the 2 μs it spins before parking. We sweep that whole window (Figure 10). Widening it is free at first, and at 25 μs the search median falls 1.6–2.1× at every tenant count and the encoders are no worse off up to eight tenants. But it is never enough, and eventually it **fails**. At ten tenants 400 μs strands 3 of 10 tenants, 1 ms does not drain, and spinning does not deploy. Grouper holds 318 μs while leaving 33% of the batch machine, beating every setting from 50 to 200 μs on both axes; the three that leave the encoders more are 7.7×, 6.6× and 4.2× slower, and at no tenant count is any setting both as fast as Grouper and as cheap. The tail is the one place a window competes, and at ten tenants 100 μs reaches 807 μs against our 891, and pays a quarter of the batch machine for it (24.3% retained against 33%) whether or not the load that justified the reservation is still arriving. Spinning is the paper's fastest arm (116 μs, flat in tenant count) and costs a core per service whether or not a request arrives, so six tenants is a ceiling here with no load term in it. Nine microservices is small, and `socialNetwork` [23] would allow four tenants with its datastores collapsed and two as it ships, so the remedy is least available exactly where the tax it addresses is worst.

This cross-section is Grouper's least favourable line, the only region of the grid where its best-effort throughput falls below Caladan's (Figure 7), and not where a peak-provisioned deployment spends its time. Below it Grouper recovers that axis (Figure 11): its best-effort deficit closes and then reverses as load falls, while every park-window setting keeps the cost it paid at the peak. At ten tenants it leaves the encoders 72% of Caladan's frames at 20,000 RPS, 124% at 10,000 and 137% at 5,000, holding the median at 318, 336 and 365 μs against Caladan's 2,445, 2,105 and 1,523. At 5,000 it leaves them more than any arm measured there, Caladan included, and at none of the three loads is a window setting both faster and cheaper; Linux is slower at all three. A reserved core's cost does not move with load, so a bursty deployment pays the *idle* figure (17% of the batch machine at two tenants, 42% at six) almost all the time for a benefit only the peak collects. Grouper's cost is proportional to the work it does.

6 Discussion and Limitations

The invariant is the system. One core per request (§3.4) rests on a synchronous datapath and a concurrency bound. Appendix C removes them one at a time; neither carries the result. Keeping the handoff and sending interior frames over the NIC still closes 93% of the median's distance from Caladan at ten tenants and 81% of the tail's; what the ring adds is that the frame and the core arrive together, so what it closes grows with the machine, from 1.10× the tail at two tenants to 1.56× at ten. Giving Caladan the bound instead does not move it, and at Grouper's own refusal rate it still costs 3.7–7.9× the latency, because a bound limits how many requests pay the per-hop acquisition and does not make a hop cheaper.

Fan-out. A group passes exactly one core along a path, yet microservices fan out. Grouper detects a branched request and donates to the branch it sent for last; the rest fall back to park-and-wake, at a cost Appendix D measures. Production analyses [45] report that past a depth of two a tier holds a single microservice with probability above 60%, which is where single-core handoff applies.

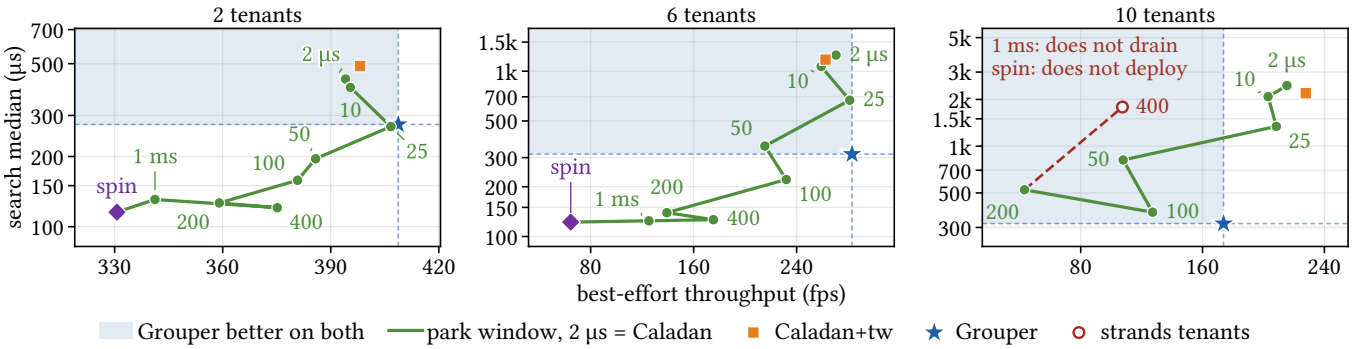


Figure 10. What each remedy costs. Search median vs. best-effort throughput at 20,000 RPS per tenant, one panel per tenant count, the park window swept and every setting labelled; shaded is where Grouper wins on both axes. Each panel is fitted to its own data, so a position is comparable within a panel and not across them. At ten tenants 400 μs strands 3 of 10 tenants.

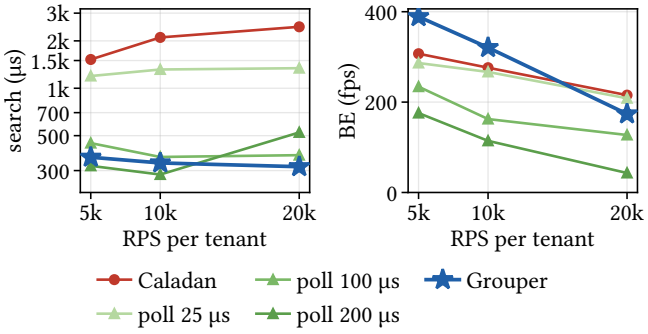


Figure 11. The same comparison as offered load falls, at ten tenants. Search median (left) and best-effort throughput (right) vs. RPS per tenant, on the same two axes as Figure 10.

Asynchronous chains. Donation requires the sender to be done with the core, which a blocking RPC is; an asynchronous send that leaves it runnable falls back to park-and-wake (§3.1). Previous studies find that microservices typically issue nested RPCs and wait synchronously for the results [66].

Sidecars. Service meshes [17, 68] put a sidecar next to every service, which under Caladan doubles the boundary crossings, and the parking tax, on every hop. Under Grouper a sidecar is another member and the handoff passes straight through it.

Portability. Grouper changes the runtime scheduler, the kernel module and the IOKernel without touching the programming model; applications are unmodified apart from the optional hook that turns a flag into a protocol-level refusal (§3.4.1). Built on Caladan, it still requires applications to use that runtime, which Junction [19] does not, and a scheduling group would fit there too.

7 Related Work

Core-granting runtimes. Kernel-bypass dataplanes [6, 15, 31, 33, 54], userspace core sizing [56] and the control-plane / data-plane split [47, 53] lead to Shenango [50] and Caladan [21] and their privileged microsecond-granularity allocator, which Junction [19] carries forward and scales to thousands of instances. All schedule *one application's* requests onto *that application's* cores, and all require the allocator to first *discover* that a core should move. Grouper keeps that machinery and removes it only from the per-hop critical path, within a trusted set of runtimes.

Making core movement faster. A parallel line makes the transfer itself cheap (Vessel [41] with MPK domains and user interrupts, HyperFlux [65] between lightweight VMs, Junction [19] with uProcs in one address space). Running a tenant's services as uProcs removes the cost measured here outright, by making the tenant a single scheduling unit with no boundary left to cross; a group makes the tenant a single allocation unit too, without merging the address spaces. Nu [58] moves the *work* instead, and Service Weaver [24] erases the boundary altogether. Each pays with the process boundary, a domain limit, or hardware such as UINTR [29, 60] commodity servers lack, making reallocation fast where a group makes the decision free. Collapsing the boundary is available to any deployment willing to give up the decomposition; a group is what is available when it is not.

Cross-domain control transfer. Directed handoff has a long lineage. LRPC [7], URPC [8], L4 [40], Mach [9] and seL4's scheduling contexts [46] all lend a thread, or the right to run, across a domain boundary [16, 48], but always inside a kernel that was itself the sole scheduler. Here a preemptive userspace allocator owns the cores, so keeping its accounting exact (§3.3) and revoking a core whose occupant it does not know (§3.6) is a different problem. It is not a futex swap [49], which moves no core, nor a ghOSt [28] or Syrup [34] policy, which cannot beat a round trip to the agent.

Microservice fast paths. Nightcore [32], SPRIGHT [55], SAND [1] and service meshes [17] accelerate the *messages* between colocated services [39], and μTune [59] shows how much of an OLDI tail per-hop threading sets. All are complementary, and we build one ourselves (§3.5), but they move the bytes while the receiver stays parked. Delivering the message *with* the core is what removes the wakeup.

Coscheduling and overload control. Coscheduling [3, 18, 51] runs communicating processes *simultaneously*; a chain is the opposite, and co-residency wastes cores in proportion to its length (§5.7). Callisto [27, 62] and Akaros [57] make granted cores first-class [13]; a group stretches that across process boundaries. Breakwater [11] and related schemes [12, 64, 67] regulate overload with RPC-granularity credits (§3.4.1).

8 Conclusion

Kernel-bypass runtimes made microsecond-scale colocation practical by harvesting cores aggressively, and that made wakeups expensive. Microservice chains pay that tax on every hop in both directions, and on a shared host it compounds, since every colocated tenant's hops arrive at the same central allocator.

The information needed to move a core is in the application's dataflow. A scheduling group lets mutually trusting services act on it, handing cores through an unprivileged kernel fast path while the allocator keeps control through reconciliation, core-addressed revocation and a pooled budget. Allocator load falls from $\Theta(R{\cdot}H)$ to $\Theta(R)$, median nearly flat and tail under a millisecond as tenants and load scale, each service still in its own address space, failure domain and release cycle.

References

[1] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. 2018. SAND: Towards High-Performance Serverless Computing. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, USA, 923–935. https://www.usenix.org/conference/atc18/presentation/akkus

[2] Thomas E. Anderson, Brian N. Bershad, Edward D. Lazowska, and Henry M. Levy. 1991. Scheduler Activations: Effective Kernel Support for the User-Level Management of Parallelism. In *Proceedings of the Thirteenth ACM Symposium on Operating Systems Principles (SOSP '91)*. Association for Computing Machinery, New York, NY, USA, 95–109. doi:10.1145/121132.121151

[3] Andrea Carol Arpaci-Dusseau. 2001. Implicit Coscheduling: Coordinated Scheduling with Implicit Information in Distributed Systems. *ACM Transactions on Computer Systems* 19, 3 (2001), 283–331. doi:10.1145/380749.380764

[4] Tom Barbette, Georgios P. Katsikas, Gerald Q. Maguire Jr., and Dejan Kostić. 2019. RSS++: Load and State-Aware Receive Side Scaling. In *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '19)*. Association for Computing Machinery, New York, NY, USA, 318–333. doi:10.1145/3359989.3365412

[5] Luiz Barroso, Mike Marty, David Patterson, and Parthasarathy Ranganathan. 2017. Attack of the Killer Microseconds. *Commun. ACM* 60, 4 (2017), 48–54. doi:10.1145/3015146

[6] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A Protected Dataplane Operating System for High Throughput and Low Latency. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, USA, 49–65. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/belay

[7] Brian N. Bershad, Thomas E. Anderson, Edward D. Lazowska, and Henry M. Levy. 1990. Lightweight Remote Procedure Call. *ACM Transactions on Computer Systems* 8, 1 (1990), 37–55. doi:10.1145/77648.77650

[8] Brian N. Bershad, Thomas E. Anderson, Edward D. Lazowska, and Henry M. Levy. 1991. User-Level Interprocess Communication for Shared Memory Multiprocessors. *ACM Transactions on Computer Systems* 9, 2 (1991), 175–198. doi:10.1145/103720.114701

[9] David L. Black. 1990. Scheduling Support for Concurrency and Parallelism in the Mach Operating System. *Computer* 23, 5 (1990), 35–43. doi:10.1109/2.53353

[10] Shuang Chen, Christina Delimitrou, and José F. Martínez. 2019. PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 107–120. doi:10.1145/3297858.3304005

[11] Inho Cho, Ahmed Saeed, Joshua Fried, Seo Jin Park, Mohammad Alizadeh, and Adam Belay. 2020. Overload Control for μs-scale RPCs with Breakwater. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Virtual Event, 299–314. https://www.usenix.org/conference/osdi20/presentation/cho

[12] Inho Cho, Ahmed Saeed, Seo Jin Park, Mohammad Alizadeh, and Adam Belay. 2023. Protego: Overload Control for Applications with Unpredictable Lock Contention. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, USA, 725–738. https://www.usenix.org/conference/nsdi23/presentation/cho

[13] Juan A. Colmenares, Gage Eads, Steven Hofmeyr, Sarah Bird, Miquel Moretó, David Chou, Brian Gluzman, Eric Roman, Davide B. Bartolini, Nitesh Mor, Krste Asanović, and John D. Kubiatowicz. 2013. Tessellation: Refactoring the OS around Explicit Resource Containers with Continuous Adaptation. In *Proceedings of the 50th Annual Design Automation Conference (DAC '13)*. Association for Computing Machinery, New York, NY, USA, 76:1–76:10. doi:10.1145/2463209.2488827

[14] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80. doi:10.1145/2408776.2408794

[15] Henri Maxime Demoulin, Joshua Fried, Isaac Pedisich, Marios Kogias, Boon Thau Loo, Linh Thi Xuan Phan, and Irene Zhang. 2021. When Idling is Ideal: Optimizing Tail-Latency for Heavy-Tailed Datacenter Workloads with Perséphone. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 621–637. doi:10.1145/3477132.3483571

[16] Dong Du, Zhichao Hua, Yubin Xia, Binyu Zang, and Haibo Chen. 2019. XPC: Architectural Support for Secure and Efficient Cross Process Call. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA '19)*. Association for Computing Machinery, New York, NY, USA, 671–684. doi:10.1145/3307650.3322218

[17] Envoy Project Authors. 2026. Envoy Proxy. https://www.envoyproxy.io/ Accessed 2026-09-02.

[18] Dror G. Feitelson and Larry Rudolph. 1992. Gang Scheduling Performance Benefits for Fine-Grain Synchronization. *J. Parallel and Distrib.*

Comput. 16, 4 (1992), 306–318. doi:10.1016/0743-7315(92)90014-E

[19] Joshua Fried, Gohar Irfan Chaudhry, Enrique Saurez, Esha Choukse, Íñigo Goiri, Sameh Elnikety, Rodrigo Fonseca, and Adam Belay. 2024. Making Kernel Bypass Practical for the Cloud with Junction. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, USA, 55–73. https://www.usenix.org/conference/nsdi24/presentation/fried

[20] Joshua Fried, Gohar Irfan Chaudhry, Enrique Saurez, Esha Choukse, Íñigo Goiri, Sameh Elnikety, Rodrigo Fonseca, and Adam Belay. 2026. Junction source repository. https://github.com/JunctionOS/junction Vendors Caladan at lib/caladan, commit 5dccc71.

[21] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. 2020. Caladan: Mitigating Interference at Microsecond Timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Virtual Event, 281–297. https://www.usenix.org/conference/osdi20/presentation/fried

[22] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. 2026. Caladan source repository. https://github.com/shenango/caladan Commit bdb4dde, May 2026.

[23] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 3–18. doi:10.1145/3297858.3304013

[24] Sanjay Ghemawat, Robert Grandl, Srdjan Petrovic, Michael Whittaker, Parveen Patel, Ivan Posva, and Amin Vahdat. 2023. Towards Modern Development of Cloud Applications. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*. Association for Computing Machinery, New York, NY, USA, 110–117. doi:10.1145/3593856.3595909

[25] Google Cloud Platform. 2026. Online Boutique (microservices-demo). https://github.com/GoogleCloudPlatform/microservices-demo Accessed 2026-09-02.

[26] gRPC Authors. 2026. gRPC: A High Performance, Open Source Universal RPC Framework. https://grpc.io/ Accessed 2026-09-04.

[27] Tim Harris, Martin Maas, and Virendra J. Marathe. 2014. Callisto: Co-scheduling Parallel Runtime Systems. In *Proceedings of the Ninth European Conference on Computer Systems (EuroSys '14)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/2592798.2592807

[28] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. 2021. ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 588–604. doi:10.1145/3477132.3483542

[29] Intel Corporation. 2026. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3 (3A, 3B, 3C & 3D): System Programming Guide*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html Order Number 325384-092US, June 2026. User interrupts (UINTR), Chapter 9; protection keys (PKU), Section 5.6.2.

[30] Calin Iorgulescu, Reza Azimi, Youngjin Kwon, Sameh Elnikety, Manoj Syamala, Vivek Narasayya, Herodotos Herodotou, Paulo Tomita, Alex Chen, Jack Zhang, and Junhua Wang. 2018. PerfIso: Performance Isolation for Commercial Latency-Sensitive Services. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, USA, 519–532. https://www.usenix.org/conference/atc18/presentation/iorgulescu

[31] Rishabh Iyer, Musa Unal, Marios Kogias, and George Candea. 2023. Achieving Microsecond-Scale Tail Latency Efficiently with Approximate Optimal Scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 466–481. doi:10.1145/3600006.3613136

[32] Zhipeng Jia and Emmett Witchel. 2021. Nightcore: Efficient and Scalable Serverless Computing for Latency-Sensitive, Interactive Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 152–166. doi:10.1145/3445814.3446701

[33] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. 2019. Shinjuku: Preemptive Scheduling for μsecond-scale Tail Latency. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, USA, 345–360. https://www.usenix.org/conference/nsdi19/presentation/kaffes

[34] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. 2021. Syrup: User-Defined Scheduling Across the Stack. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 605–620. doi:10.1145/3477132.3483548

[35] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2019. Datacenter RPCs Can Be General and Fast. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, USA, 1–16. https://www.usenix.org/conference/nsdi19/presentation/kalia

[36] Martin Karsten and Saman Barghi. 2020. User-level Threading: Have Your Cake and Eat It Too. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 1 (2020), 17:1–17:30. doi:10.1145/3379483

[37] Georgios P. Katsikas, Tom Barbette, Dejan Kostić, Rebecca Steinert, and Gerald Q. Maguire Jr. 2018. Metron: NFV Service Chains at the True Speed of the Underlying Hardware. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, USA, 171–186. https://www.usenix.org/conference/nsdi18/presentation/katsikas

[38] Jacob Leverich and Christos Kozyrakis. 2014. Reconciling High Server Utilization and Sub-millisecond Quality-of-Service. In *Proceedings of the Ninth European Conference on Computer Systems (EuroSys '14)*. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/2592798.2592821

[39] Bojie Li, Tianyi Cui, Zibo Wang, Wei Bai, and Lintao Zhang. 2019. SocksDirect: Datacenter Sockets Can Be Fast and Compatible. In *Proceedings of the ACM SIGCOMM 2019 Conference*. Association for Computing Machinery, New York, NY, USA, 90–103. doi:10.1145/3341302.3342071

[40] Jochen Liedtke. 1993. Improving IPC by Kernel Design. In *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles (SOSP '93)*. Association for Computing Machinery, New York, NY, USA, 175–188. doi:10.1145/168619.168633

[41] Jiazhen Lin, Youmin Chen, Shiwei Gao, and Youyou Lu. 2024. Fast Core Scheduling with Userspace Process Abstraction. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 280–295. doi:10.1145/3694715.3695976

[42] David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. 2015. Heracles: Improving Resource Efficiency at Scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA '15)*. Association for Computing Machinery, New York, NY, USA, 450–462. doi:10.1145/2749469.2749475

[43] Locust Authors. 2026. Locust: An Open Source Load Testing Tool. https://locust.io/ Accessed 2026-09-04.

[44] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. 2016. The Linux Scheduler: A Decade of Wasted Cores. In *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys '16)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/2901318.2901326

[45] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. 2021. Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 412–426. doi:10.1145/3472883.3487003

[46] Anna Lyons, Kent McLeod, Hesham Almatary, and Gernot Heiser. 2018. Scheduling-Context Capabilities: A Principled, Light-Weight Operating-System Mechanism for Managing Time. In *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3190508.3190539

[47] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Michael Dalton, Nandita Dukkipati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Erik Rubow, Michael Ryan, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. 2019. Snap: A Microkernel Approach to Host Networking. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*. Association for Computing Machinery, New York, NY, USA, 399–413. doi:10.1145/3341301.3359657

[48] Zeyu Mi, Dingji Li, Zihan Yang, Xinran Wang, and Haibo Chen. 2019. SkyBridge: Fast and Secure Inter-Process Communication for Microkernels. In *Proceedings of the Fourteenth EuroSys Conference (EuroSys '19)*. Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3302424.3303946

[49] Peter Oskolkov. 2022. User Managed Concurrency Groups (UMCG) and the FUTEX_SWAP Patch Series. https://lwn.net/Articles/879398/ Linux kernel patch series, 2021–2022. Accessed 2026-09-02.

[50] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. 2019. Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, USA, 361–378. https://www.usenix.org/conference/nsdi19/presentation/ousterhout

[51] John K. Ousterhout. 1982. Scheduling Techniques for Concurrent Systems. In *Proceedings of the 3rd International Conference on Distributed Computing Systems (ICDCS '82)*. IEEE Computer Society, Miami/Ft. Lauderdale, FL, USA, 22–30. https://web.stanford.edu/~ouster/cgi-bin/papers/coscheduling.pdf

[52] Aleksey Pesterev, Jacob Strauss, Nickolai Zeldovich, and Robert T. Morris. 2012. Improving Network Connection Locality on Multicore Systems. In *Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys '12)*. Association for Computing Machinery, New York, NY, USA, 337–350. doi:10.1145/2168836.2168870

[53] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. 2014. Arrakis: The Operating System is the Control Plane. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, USA, 1–16. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/peter

[54] George Prekas, Marios Kogias, and Edouard Bugnion. 2017. ZygOS: Achieving Low Tail Latency for Microsecond-scale Networked Tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*. Association for Computing Machinery, New York, NY, USA, 325–341. doi:10.1145/3132747.3132780

[55] Shixiong Qi, Leslie Monis, Ziteng Zeng, Ian-Chin Wang, and K. K. Ramakrishnan. 2022. SPRIGHT: Extracting the Server from Serverless Computing! High-performance eBPF-based Event-driven, Shared-memory Processing. In *Proceedings of the ACM SIGCOMM 2022 Conference*. Association for Computing Machinery, New York, NY, USA, 780–794. doi:10.1145/3544216.3544259

[56] Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John Ousterhout. 2018. Arachne: Core-Aware Thread Management. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, USA, 145–160. https://www.usenix.org/conference/osdi18/presentation/qin

[57] Barret Rhoden, Kevin Klues, David Zhu, and Eric Brewer. 2011. Improving Per-Node Efficiency in the Datacenter with New OS Abstractions. In *Proceedings of the 2nd ACM Symposium on Cloud Computing (SOCC '11)*. Association for Computing Machinery, New York, NY, USA, 1–8. doi:10.1145/2038916.2038941

[58] Zhenyuan Ruan, Seo Jin Park, Marcos K. Aguilera, Adam Belay, and Malte Schwarzkopf. 2023. Nu: Achieving Microsecond-Scale Resource Fungibility with Logical Processes. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, USA, 1409–1427. https://www.usenix.org/conference/nsdi23/presentation/ruan

[59] Akshitha Sriraman and Thomas F. Wenisch. 2018. μTune: Auto-Tuned Threading for OLDI Microservices. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, USA, 177–194. https://www.usenix.org/conference/osdi18/presentation/sriraman

[60] Jovan Stojkovic, Chunao Liu, Muhammad Shahbaz, and Josep Torrellas. 2025. HardHarvest: Hardware-Supported Core Harvesting for Microservices. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 708–722. doi:10.1145/3695053.3731071

[61] Gil Tene. 2026. wrk2: A Constant Throughput, Correct Latency Recording Variant of wrk. https://github.com/giltene/wrk2 Accessed 2026-09-04.

[62] Andrew Tucker and Anoop Gupta. 1989. Process Control and Scheduling Issues for Multiprogrammed Shared-Memory Multiprocessors. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP '89)*. Association for Computing Machinery, New York, NY, USA, 159–166. doi:10.1145/74850.74866

[63] VideoLAN Organization. 2026. x264. https://www.videolan.org/developers/x264.html Accessed 2026-09-02.

[64] Matt Welsh, David Culler, and Eric Brewer. 2001. SEDA: An Architecture for Well-Conditioned, Scalable Internet Services. In *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles (SOSP '01)*. Association for Computing Machinery, New York, NY, USA, 230–243. doi:10.1145/502034.502057

[65] Yibo Yan and Seo Jin Park. 2026. Offering Microsecond-Scale Cross-VM Core Elasticity on Colocated Lightweight Virtual Machines. doi:10.48550/arXiv.2608.12633 arXiv:2608.12633 [cs.DC] Version 1, 12 August 2026.

[66] Arash Pourhabibi Zarandi, Mark Sutherland, Alexandros Daglis, and Babak Falsafi. 2021. Cerebros: Evading the RPC Tax in Datacenters. In *MICRO '21: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. Association for Computing Machinery, New York, NY, USA, 407–420. doi:10.1145/3466752.3480055

[67] Hao Zhou, Ming Chen, Qian Lin, Yong Wang, Xiaobin She, Sifan Liu, Rui Gu, Beng Chin Ooi, and Junfeng Yang. 2018. Overload Control for Scaling WeChat Microservices. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '18)*. Association for Computing Machinery, New York, NY, USA, 149–161. doi:10.1145/3267809.3267823

[68] Xiangfeng Zhu, Guozhen She, Bowen Xue, Yu Zhang, Yongsu Zhang, Xuan Kelvin Zou, Xiongchun Duan, Peng He, Arvind Krishnamurthy, Matthew Lentz, Danyang Zhuo, and Ratul Mahajan. 2023. Dissecting

Overheads of Service Mesh Sidecars. In *Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23)*. Association for Computing Machinery, New York, NY, USA, 142–157. doi:10.1145/3620678.3624652

A Implementation size

Table 5 counts lines added and lines removed against the Caladan branch point from which Grouper was developed (commit `bdb4dde`).

Table 5. Implementation size (lines added / removed against the branch point).

component	added	removed
`ksched` kernel module	391	1
IOKernel (control, `sched.c`, `ias.c`, directpath, stats)	4,181	83
runtime scheduler, kthreads, net stack	3,609	111
runtime loopback datapath (`sched_group_lb.c`)	975	0
shared headers (`inc/`, `base/`)	1,403	8
total	**10,559**	**203**

B Fixes to the baseline

Stock Caladan was taken at its latest commit at the time (`bdb4dde`). Several defects were fixed prior to measuring, and every Caladan number in this paper includes them. The primary fix was in the IOKernel's idle fast-wake path, which checks for arriving packets between periodic allocation passes. Stock Caladan evaluated arrivals by calculating descriptor age against a device clock refreshed only in the slow pass; newly arrived packets registered as zero delay and missed immediate wakeups. Testing the descriptor's completion parity bit directly detects work without clock overhead, restoring prompt wakeups. The remaining fixes address concurrency races and buffer exhaustion under heavy multi-tenant load. Publication races during TCP connection setup caused spurious timeouts, stalled receive pollers and buffer pool deadlocks from deferred transmit completions and pinned receive strides. With these fixes, stock Caladan runs stably, without artificial stalls or drops.

C Ablations

The invariant of §3.4 rests on two mechanisms. Removing either leaves most of Grouper's advantage intact. The bound's size is a third question of how many lanes and what that spends, not a third mechanism. All three experiments keep §5.1's configuration otherwise, at three interleaved repetitions.

C.1 The shared-memory datapath

`groupwire` keeps the scheduling group entire (the handoff, the lanes, the pinned receive queues, the pooled budget) and sends interior frames over the NIC, naming the destination pad in the IP header so that a frame still lands on the kthread its sender donated to.

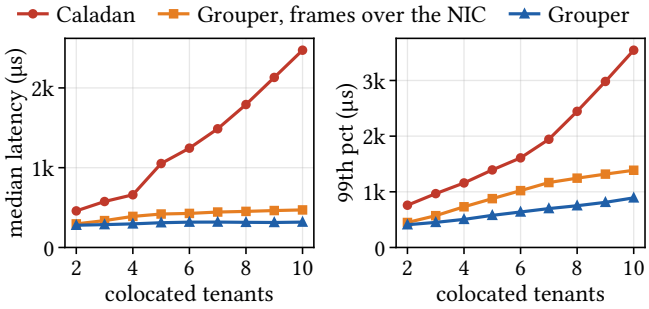


Figure 12. What the ring is worth. Full-depth `search` median (left) and 99th percentile (right) at 20,000 RPS per tenant. `groupwire` is Grouper with interior frames sent over the NIC instead of through the peer's ring; nothing else about that group changes.

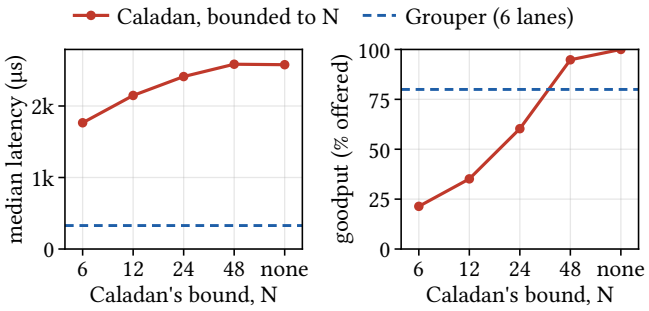


Figure 13. No bound places Caladan where Grouper is. Stock Caladan bounded to N requests in flight per deployment, at ten tenants and 30,000 RPS each; N=6 is Grouper's own lane count and none is unbounded Caladan. At no N is Caladan below Grouper's level on the left *and* above it on the right.

Most of Grouper remains without the ring (Figure 12). At ten tenants the handoff alone closes 93% of the median's distance from Caladan and 81% of the tail's. What it does not close grows with the machine. The wire arm's tail is 1.10× the ring's at two tenants and 1.56× at ten, because the frame and the core are then dispatched separately and *drift*. The request waits for whichever of the two is later. Moving the bytes is worth about a microsecond; moving them *with* the core is worth the rest.

C.2 The concurrency bound

Grouper runs one request per lane and refuses the rest at the edge, so a question §5 leaves open is where Caladan then lands when it is bounded the same way. `capN` is stock Caladan bounded to N requests in flight per deployment,

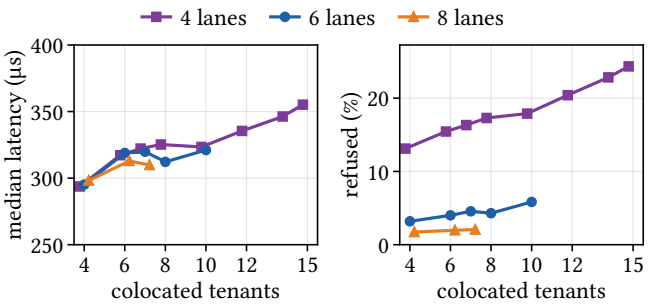


Figure 14. The hop does not shift with the bound. Full-depth search median (left) and fraction refused (right) at 20,000 RPS per tenant. A line that continues further right packs more tenants. Six is §5.1's setting.

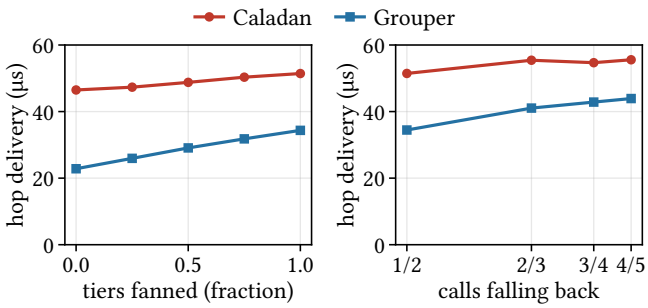


Figure 15. Fan-out costs Grouper its donations, not its advantage. Delivery time of one interior hop. How many of the four tiers fan on the left, at degree two. How wide every tier fans on the right, as the fraction of a tier's calls that fall back onto park-and-wake.

refused at the frontend with the same reply a lane denial produces.

No bound gets Caladan there (Figure 13). At six permits against six lanes (ten tenants, 30,000 RPS), the two run the same concurrency, 5.8 requests in flight against 5.5, and Caladan is 5.4× slower at the median while refusing 78.6% against 20.0%; at Grouper's own refusal rate it costs 3.7–7.9× the latency across the grid. Little's law says why, from residency alone. An interior hop costs `cap6` 183.9 μs against 19.1, which over ≈5.6 RPCs is essentially the whole of Caladan's 912 μs residency against 230. The same six permits carry four times the traffic because what occupies them is 9.6× cheaper. Three of that arm's effects favour Caladan. A bounded deployment loads the machine less, so its own hop gets 13–29% cheaper; a refusal is issued when the system is momentarily full, so a tight cap serves requests drawn from calmer instants; and `cap6` leaves the encoders more frames than Grouper while delivering a fifth of the traffic.

C.3 The lane count

A request holds one lane for its residency, and there are as many lanes as a member has kthreads, so the lane count is the group's concurrency bound and its peak core claim together (§3.4). §5 measures one setting of that knob (six) and Appendix C.2 asks whether the bound is the result. This asks what that six is (Figure 14).

Grouper only, at §5's 20,000 RPS cross-section, where six lanes refuse 3.3–5.7%. A cell that would book more cores at peak occupancy than the machine has, after the spinning load generator, is not run. The IOKernel leaves 70 to allocate, eight of them guaranteed to the generator, 62 left for LC, so four lanes reach 15 tenants, six reach 10 and eight reach 7.

The hop itself does not move (Figure 14, left). At 7 tenants the full-depth search median is 322, 320 and 310 μs at four, six and eight lanes. What moves is the bound (right). Four lanes refuse 16.3%, six 4.6%, eight 2.1%. The same four lanes at 15 tenants (a machine six cannot occupy) refuse 24.3% and serve 227,000 requests a second against eight lanes at 7 tenants serving 137,000 and refusing 2.1%. Fewer lanes pack more tenants and refuse more of each; more lanes do the reverse. Six is a point on that tradeoff.

D Fan-out

A tier of degree R spends R donations when it calls serially and one when it fans, so $R-1$ of R fall back to ordinary park-and-wake. Figure 15 sweeps both axes on a synthetic graph of four tiers over nine services at a fixed depth of five, varying how many tiers fan, at degree two, and how wide every tier fans. A hop costs 22.8 μs with nothing fanning, 34.4 μs with all four tiers fanned, and 43.9 μs at degree five, where four calls in five fall back to ordinary park-and-wake, against Caladan's 46.5–55.6 μs, unmoved because it pays an allocator wake either way. The cost rises and then saturates without crossing. The chain call and every reply leg are still handed over directly, whatever the width. Detection is exact, firing 1.02 times per request per fanned tier and 0.04 times on the sequential graph, and the fallback costs the batch task nothing extra. Best-effort throughput stays within 2% of Caladan's at every point on both axes.

The settings match §5.1's but for three. They use two tenants at 2,000 RPS each over twelve lanes a service, rather than ten at 20,000 over six. A fanned request holds its lane until its widest branch returns, so at §5's cross-section the refusal rate would rise with the fanned fraction and the x axis would be admission control rather than fan-out; here the group refuses under 0.04% of requests at each point.